RESEARCH AND IMPLEMENTATION REVIEW

New Lean proof simplifications

After the existing roadmap

`Computability/DiophantineConsistency`

Prepared for Vladimir Reshetnikov

26 October 2026

Pinned source snapshot

`9bd29079d5e1c6cc1c0bcaee6cb57d59b835cba9`

Validation environment

Lean 4.32.0; mathlib `81a5d257c8e410db227a6665ed08f64fea08e997`.

Main finding. The most useful remaining opportunities remove mathematical or semantic obligations: delete two unnecessary internal inductions; derive a negative Pell obstruction from an existing positive-unit classification; transport three residue obstructions through one common discriminant; and extract heterogeneous block minima by one finite-sum theorem. A fifth, lower-priority design packages monotone certificate closure.

Every earlier idea in the six placed proposal packets and five incoming archives is treated as already applied. Kernel-checked mechanisms, checks against current source, and integration work still needed are distinguished throughout.

This report proposes changes. It does not claim a merged repository refactor, a full-project build, a new consistency theorem, or a smaller Diophantine equation.

Contents

1 Results, scope, and the counterfactual baseline **2**
1.1 What "already applied" means here . . . 2
1.2 How evidence and savings are reported . . . 3

2 Delete internal inductions whose hypotheses are unused **3**
2.1 Two concrete targets . . . 3
2.2 The internal-model proof is preserved . . . 3
2.3 Why this is additional to the earlier proposals . . . 4
2.4 The import-layer issue is real . . . 4

3 Derive the negative Pell obstruction from positive units **5**
3.1 The duplicated mathematical infrastructure . . . 5
3.2 A small congruence invariant . . . 5
3.3 Square a hypothetical negative unit . . . 5
3.4 Dependency arrangement and validation . . . 6

4 Normalize three residue obstructions to one existing theorem **7**
4.1 The common arithmetic statement . . . 7
4.2 Four clients, one residue certificate . . . 7
4.3 Why this is not another proposal for finite checking . . . 7

5 Extract all block equalities from one minimum-saturation theorem **8**
5.1 The target has coupled and nonzero minima . . . 8
5.2 The generic principle is an existing library theorem . . . 8
5.3 What is gained, and what is not . . . 9

6 Optional: a calculus for monotone local certificates **9**
6.1 The remaining source pattern . . . 9
6.2 An exact abstract interface . . . 10
6.3 Mapping it to the actual model . . . 10
6.4 Limits and the adoption decision . . . 10

7 Integration sequence and evidence discipline **11**
7.1 A practical order . . . 11
7.2 What the validation artifacts mean . . . 11
7.3 Measurements and counterfactual honesty . . . 12

8 Artifact guide and reproduction **12**

1 Results, scope, and the counterfactual baseline

There are four concrete recommendations and one optional abstraction. The strongest conceptual change is the Pell proof: an independent descent can be replaced by a congruence consequence of the positive-unit theorem already used elsewhere in the project. The most direct semantic change is smaller: two internal inductions never use their induction hypotheses, so their case decomposition is sufficient.

	Proposal	Obligation removed	Status and priority
1	Internal zero/successor cases	Two stage formulas, their satisfaction plumbing, and induction transport for zero sums/products.	Checked semantic mechanism; exact patches supplied; object-module integration remains. High value.
2	Positive Pell units imply the negative obstruction	A second, separate well-founded descent and its transformed-coordinate estimates.	Kernel-checked complete arithmetic replacement. Move an existing foundation to a lower module. High value.
3	One discriminant obstruction	Three independent copies of the same modulo-four contradiction.	Checked specializations of the actual existing source theorem. Low integration cost.
4	Equality at block minima	Repeated nonlinear extraction of five equality conditions from one aggregate equality.	Kernel-checked exact theorem and source-level replay evidence. Low integration cost.
5	Monotone certificate closure	Repeated proofs for empty, union, insertion, and extraction of a certified value.	Axiom-free generic prototype. Adapter and cost justification remain. Optional.

1.1 What "already applied" means here

The exclusion audit includes six packets under `research/lean-proof-simplification` and five ZIPs under `docs/incoming`. The audit records 323 placed files and 400 archive members, with no nested archives. Its 56 grouped mechanisms include optional ideas in notes, proposals marked pending or deferred, and designs not previously compiled. These counts describe the exclusion corpus, not the number of independent mathematical contributions.

Consequently, this report does not recommend scanner folds, decoder-graph fusion, assignment naturality, generated power certificates, a shared sum-of-squares interface, signed Pell congruences, rational-root preservation, internal recursion fusion, or max-plus depth automation as new work. The previous proposals also contain less visible suggestions about support-restricted evaluation, renaming-chain normalization, and cyclic packing as an algebra homomorphism. Those are excluded too. The complete matrix is supplied as `evidence/prior_proposals_audit.md`.

A new helper name or a further example of an already proposed interface is insufficient. Each

accepted recommendation removes an obligation the old interface would leave in place. This is a scoped nonduplication finding relative to the supplied repository corpus, not a claim that the underlying mathematics is new in the literature.

1.2 How evidence and savings are reported

The source snapshot is fixed at `9bd29079d5e1c6cc1c0bcaee6cb57d59b835cba9`. The complete hypothetical tree obtained by implementing every earlier proposal does not exist as a unique artifact: several earlier designs offer alternatives. We therefore do not add up raw line reductions and call them incremental savings after that hypothetical migration.

Instead, the report makes two separate claims. First, each selected mechanism remains applicable after the preceding roadmap. Second, where a concrete patch is supplied, its observed source reduction or eliminated proof calls is measured against the pinned snapshot. Shared helpers, moved code, imports, and uncompiled adapters are included in the discussion. A shorter source file is not evidence of faster elaboration or a smaller proof term.

All recorded Lean checks use the repository's exact Lean and mathlib versions. Standalone prototypes are clearly distinguished from full-module replays. The archive's validation receipts and logs are authoritative about the exact scope. No whole-repository rebuild is claimed, and no change was pushed to GitHub.

2 Delete internal inductions whose hypotheses are unused

2.1 Two concrete targets

In `ObjectAdditionCancellation.addRel_zero_iff`, the successor branch of an `omega_ind` proof begins with `intronhn_`. The discarded argument is the induction hypothesis. The branch proves that a sum with a successor second argument cannot be zero, independently of any statement about the preceding argument.

The same pattern occurs in `ObjectNaturalZero.mulRel_zero_iff`. Its successor branch uses totality and functionality of multiplication, the successor equation, and the previously established zero-sum theorem. It does not use the induction hypothesis. The supporting private declarations are `zeroSumStage`, `zeroSumStage_spec`, `zeroProductStage`, and `zeroProductStage_spec`.

The important observation is logical. A proof of $P(0)$ together with a proof of $P(s(n))$ that does not assume $P(n)$ is a proof by cases whenever every natural is zero or a successor. There is no remaining inductive invariant to encode.

2.2 The internal-model proof is preserved

Fix an arbitrary model carrier V and a set-theoretic axiom package H. Write ω^H for its internal natural numbers, 0^H for its empty set, and s^H for its successor. The existing theorem `nat_zero_or_succ` has precisely the required form:

$$y \in \omega^H \implies y = 0^H \ \vee \ \exists n \in \omega^H,\ y = s^H(n).$$

This statement ranges over the entire internal omega. It does not assert that y is the interpretation of an external Lean numeral.

For addition, assume $\mathrm{AddRel}_H(x, y, 0^H)$. Its domain facts supply $y \in \omega^H$. Split y using the displayed theorem.

1. If $y = 0^H$, functionality compares the supplied addition result with `addRel_zero` and gives $x = 0^H$.
2. If $y = s^H(n)$, choose b with $\mathrm{AddRel}_H(x, n, b)$ by totality. The successor law produces
$$\mathrm{AddRel}_H(x, s^H(n), s^H(b)).$$
Functionality gives $0^H = s^H(b)$, contradicted by $b \in s^H(b)$ and the empty-set membership theorem.

The reverse implication is unchanged.

For multiplication, assume $x, y \in \omega^H$ and $\mathrm{MulRel}_H(x, y, 0^H)$. The zero case is immediate. In the successor case choose the predecessor product b. The successor law identifies the result with $b + x$; the zero-sum theorem forces $x = 0^H$. Again, no statement about the predecessor product being zero is needed.

2.3 Why this is additional to the earlier proposals

Earlier `omega_ind_of_spec`-style interfaces, constructor views, and internal recursive-graph fusion shorten a genuine induction or its satisfaction transport. Applied to these two targets, they would still leave an unnecessary induction. The present change deletes the induction and the formula needed solely to express its predicate. A broader global search for unused hypotheses is not the contribution; the two concrete semantic arguments above are.

The genuine cancellation induction in `addRel_cancel_right` must remain. Its successor branch uses the induction hypothesis after cancelling successors. The proposed patch removes only the zero-sum and zero-product stages.

2.4 The import-layer issue is real

`nat_zero_or_succ` currently lives in `BoundedZFCConsistency.InternalSat`. Neither target currently imports that module. A minimal patch can add this import to `ObjectAdditionCancellation`, making it available transitively to `ObjectNaturalZero`. That is a concrete implementation option, but it brings a higher semantic layer into foundational arithmetic.

The preferable production arrangement is to move the existing zero/successor block, with its formula and specification, into a low-level `BoundedZFCConsistency.NaturalCases` module importing `OmegaRecursion` and `CodePredicate`. Both are already in the arithmetic dependency closure. `InternalSat` and the arithmetic module then import it. The public theorem name and proof remain unchanged. This is relocation of an existing result, not a third proof of natural-number decomposition.

The preferred four-file extraction patch is 30 physical lines and 1,238 UTF-8 bytes smaller than the raw pinned sources, including the new module and import overhead. The minimal heavy-import alternative removes 41 lines and 1,626 bytes. These are alternative patches, not additive reductions. Both pass textual `git apply --check`; the semantic core is kernel checked with empty axiom lists. The current object-module closure was not rebuilt in this review; the two import arrangements therefore remain integration candidates. In particular, a checked theorem over an explicit semantic interface is evidence for the reasoning, not a claim that every repository import and elaboration detail has been validated.

3 Derive the negative Pell obstruction from positive units

3.1 The duplicated mathematical infrastructure

`ConsecutivePell.norm_ne_neg_one` proves, for integers n, x, y with $n \geq 2$,

$$x^2 - n(n+1)y^2 \neq -1. \tag{1}$$

Its current private proof is a strong induction on the natural absolute value of the second coordinate. It constructs an inverse-unit transform, proves that the transformed second coordinate is positive and smaller, verifies preservation of the norm, and converts the transformed integer back to a natural induction index.

Elsewhere, `NativePellBootstrap.firstUnit_fundamental` already proves that

$$\varepsilon_n = (2n+1) + 2\sqrt{n(n+1)}$$

is the fundamental positive norm-one unit for $n > 0$. Mathlib's `Pell.IsFundamental.eq_pow_of_nonneg` classifies every norm-one solution with positive first coordinate and nonnegative second coordinate as a natural power of this unit.

The new proof uses that existing classification to derive (1). It does not introduce another classification theorem or repeat the old signed-congruence proposal.

3.2 A small congruence invariant

Put $D = n(n+1)$ and write

$$\varepsilon_n^k = X_k + Y_k\sqrt{D}.$$

The multiplication law gives

$$X_{k+1} = (2n+1)X_k + 2n(n+1)Y_k.$$

Since $X_0 = 1$, induction immediately yields

$$2n \mid X_k - 1. \tag{2}$$

The induction is a short algebraic argument on an external exponent k in the already available classification, not a descent on an arbitrary candidate solution. If $X_k - 1 = 2nt$, an explicit quotient for $X_{k+1} - 1$ is

$$t + X_k + (n+1)Y_k.$$

That quotient is exactly the witness used by the checked Lean proof. Here X_k is the first coordinate; the field trace would be $2X_k$.

3.3 Square a hypothetical negative unit

Proposition 3.1. *Let $n, x, y \in \mathbb{Z}$ and $n \geq 2$. If every norm-one solution for $D = n(n+1)$ with positive first coordinate and nonnegative second coordinate has first coordinate congruent to 1 modulo $2n$, then (1) holds.*

Proof. Suppose $x^2 - Dy^2 = -1$. Then

$$U = 2x^2 + 1, \qquad V = |2xy|$$

satisfy $U > 0$, $V \geq 0$, and

$$U^2 - DV^2 = 1.$$

The norm identity follows by expanding and using $x^2+1 = Dy^2$. Apply the positive-unit classification and (2) to obtain

$$2n \mid U - 1 = 2x^2.$$

On the other hand,

$$2x^2 + 2 = 2n(n+1)y^2.$$

Subtracting the two divisibilities gives $2n \mid 2$. Because $n \geq 2$, the positive divisor $2n$ exceeds 2, a contradiction. □

This proof handles both signs of x and y through the squaring step. It does not need a new sign-case interface. The natural-number parameter expected by the existing fundamental-unit theorem is obtained from $n \geq 2$ using Lean's checked integer-to-natural lift.

3.4 Dependency arrangement and validation

Move `firstUnit` and `firstUnit_fundamental`, unchanged, into a small common arithmetic module. Both the native bootstrap and the negative-norm theorem can import it. Add the first-coordinate congruence there or in an adjacent module. This avoids making the elementary negative-norm result depend on the native source-register development.

The source graph inspected in this review did not reveal an existing path from `NativePellBootstrap` back to `ConsecutivePell`. Thus the issue is separation of dependencies, not an observed import cycle. Moving the common foundation also reduces the chance of a future cycle as the native development grows.

The supplied prototype copies the existing fundamental-unit proof solely to make the arithmetic experiment standalone. The actual proposed `ConsecutivePellUnit` and replacement `ConsecutivePell` modules were also compiled under their production namespaces, without reusing old project build artifacts. In production that code should be moved, not duplicated. The new congruence and complete replacement proof compile with Lean 4.32.0. The resulting theorem uses the standard axioms `propext`, `Classical.choice`, and `Quot.sound`; no `sorryAx` appears. The log and source hashes are included.

Counting nonblank declaration and proof lines, with statements included and comments/imports excluded, the old descent plus public wrapper has 49 lines; the new congruence helper, classification wrapper, and public proof have 33. This is a 16-line local reduction (about 33 percent). The 34 lines of existing unit definition and fundamental proof are moved unchanged and are counted as neither new code nor savings. The full edited `NativePellBootstrap` and the entire project were not rebuilt.

The gain is removal of the independent descent and its local estimates. It is not a measured build-time improvement, and it does not change the arithmetic statement, the polynomial, or its witness count. An application importing only the old elementary module could experience a larger import closure; that tradeoff should be measured before adopting the new dependency structure.

4 Normalize three residue obstructions to one existing theorem

4.1 The common arithmetic statement

The useful normal form is

$$X^2 - (A^2 - 1)Y^2 \neq -1 \qquad (A, X, Y \in \mathbb{Z}). \tag{3}$$

It has no positivity premise. Modulo four, A^2 is 0 or 1. If it is 1, the left side is X^2, which is not -1. If it is 0, the left side is $X^2 + Y^2$, whose possible residues are $0, 1, 2$, never 3.

The existing `ShortEquation.strong_ne_neg_one` already proves the shifted form

$$X^2 - (a^2 + 4a + 3)Y^2 \neq -1.$$

Because $a^2 + 4a + 3 = (a + 2)^2 - 1$, setting $a = A - 2$ gives (3). There is no need to pay for a new residue-checking engine or another shared arithmetic axiom.

4.2 Four clients, one residue certificate

Existing target	Substitution into the common discriminant
`ShortEquation.strong_ne_neg_one`	$A = a + 2$, $X = x$, $Y = y$. Keep this existing certificate as the shared source.
`ShortSystem.square_aux_ne_neg_one`	$A = t$, $X = th$, $Y = y$. Ring normalization identifies t^2h^2 with $(th)^2$.
`MinimumNorm.main_norm_ne_neg_one`	Exactly the shifted theorem already in `ShortEquation`.
`MinimumNorm.square_auxiliary_ne_neg_one`	$A = I$, $X = IV$, $Y = V + e$, after unfolding the auxiliary norm.

For the second client, the replacement has the shape

```
convert ShortEquation.strong_ne_neg_one (t - 2) (t * h) y
  using 1 <;> ring
```

The archive contains checked code with the actual signatures. For the fourth client, the algebraic identity is

$$\mathrm{auxiliary}(I^2, V, e) = I^2V^2 - (I^2 - 1)(V + e)^2 = (IV)^2 - (I^2 - 1)(V + e)^2.$$

The first equality is the source definition, so the proof has a small, transparent normalization boundary.

4.3 Why this is not another proposal for finite checking

The finite modulo-four proof already exists and remains. The new step is recognizing that several differently presented norm expressions instantiate the same theorem through an affine change of discriminant parameter and a multiplicative change of the first coordinate. Generic advice to use `decide`, share finite tables, or search for theorem subsumption is not the claim.

After the change, all four public results have one arithmetic obstruction as their source of truth. The three adapters are ring identities with no extra hypotheses. In particular, no sign restriction on t, h, I, V, e, or $V + e$ is introduced. The code does not confuse a square of a product with a square root requiring a nonnegative choice.

The existing `ShortEquation.lean` was recompiled from the pinned source in the isolated replay. The three specializations were checked against that actual theorem, rather than only against a hand-transcribed surrogate. The obstruction-only patch removes 12 physical lines across `MinimumNorm` and `ShortSystem`, including its added import. The complete modified `MinimumNorm` module was compiled using unchanged `ConsecutivePell` and `ShortEquation` sources rebuilt from the pinned snapshot. The replacement `ShortSystem` proof was checked against that real theorem, but the full `ShortSystem` module was not rebuilt. A separate unshifted-normal-form prototype is also supplied for exposition. It is an alternative presentation, not another helper that must be added in production.

5 Extract all block equalities from one minimum-saturation theorem

5.1 The target has coupled and nonzero minima

`MinimumNorm.minima76_zero_iff` assumes lower bounds and exact equality conditions for three norm blocks. Its aggregate expression is

$$M = n_0^2 + n_1^2 + n_a^2 + W^2 + (n_k - 1)^2 + r_t^2 - 3.$$

The current proof separately asks nonlinear arithmetic to deduce five equality statements from $M = 0$. The relevant blocks and minima are

$$(f_i)_{i<5} = (n_0^2,\ n_1^2,\ n_a^2 + W^2,\ (n_k - 1)^2,\ r_t^2), \qquad (m_i)_{i<5} = (1, 1, 1, 0, 0).$$

Notice that $n_a^2 + W^2$ is one block. Treating its two summands as independent unit minima would be false and would lose the actual argument establishing its equality case.

5.2 The generic principle is an existing library theorem

Lemma 5.1 (Equality at lower bounds)**.** *For a finite index type and integer families $m_i \le f_i$,*

$$\sum_i f_i = \sum_i m_i \quad \Longleftrightarrow \quad \forall i,\ f_i = m_i.$$

If $f_i = m_i \leftrightarrow P_i$, the right side can be replaced by $\forall i, P_i$.

Proof. Each difference $f_i - m_i$ is nonnegative. Their sum is zero exactly when every difference is zero. The reverse implication is substitution. Lean already provides this result in `Finset.sum_eq_sum_iff_of_le`. □

Apply this theorem once to the five displayed blocks. Normalize the finite sum and finite universal quantifier. Rewrite the first three equality conditions using the supplied hypotheses, and the last two using `sq_eq_zero_iff` and `sub_eq_zero`. The result is exactly

$$n_0 = 1 \ \wedge\ n_1 = 1 \ \wedge\ n_a = 1 \ \wedge\ W = 0 \ \wedge\ n_k = 1 \ \wedge\ r_t = 0.$$

The public theorem and its hypotheses are unchanged.

5.3 What is gained, and what is not

The proof no longer performs five separate nonlinear consequence searches. It states the five lower bounds once, invokes a finite-sum equality theorem, and exposes the equality conditions. This improves the alignment between the mathematical proof and the source. In the direct prototype, the lower-bound vector is local; adding a permanent project-wide structure is unnecessary.

This differs from the previously proposed common zero-sum-of-squares lemma. Here the lower bounds are heterogeneous, three are nonzero, and one is a coupled block whose equality condition is a conjunction. Applying a zero-sum theorem to residuals $f_i - m_i$ is of course a mathematical proof of the lemma, but the new source interface packages the actual minimum certificates and their equality cases without rediscovering those residual facts in each client.

The analogous `minima75_zero_iff` already derives its result from `minima76_zero_iff` by substituting $(n_k n_t, 0)$ for the last two arguments. It therefore inherits the new proof automatically; no second migration is needed and no independent source reduction is claimed for it. The generic semantic form also accommodates later finalizers with different known minima or coupled arithmetic constraints, once a concrete additional client is identified.

The supplied `MinimaSaturation.lean` checks the exact existing `minima76` expression, theorem statement, and replacement body. It includes a reusable statement with equality predicates as a mechanism demonstration. The checked full-module patch keeps one small private wrapper. Its theorem body falls from 13 to 9 lines and its five `nlinarith` calls disappear. Including that helper and the explicit imports, however, the one-client patch grows by 7 physical lines. This is a simplification of reasoning and a reusable boundary, not a net source-size win yet. The standalone direct-library version shows an alternative that avoids the wrapper; it should be replayed in the target module before choosing it for a size-oriented migration. The combined obstruction-plus-saturation patch changes the two affected files from 866 to 861 physical lines, a net reduction of 5. These measurements are not added to the Pell or internal-model measurements, which use different scopes or counting rules.

6 Optional: a calculus for monotone local certificates

6.1 The remaining source pattern

`ObjectCodeBound.BoundClosed` says that every encoded pair (c, b) in a model-set certificate S has a natural result b and obeys a local constructor equation whose dependencies are also found in S. The module separately proves closure for the empty certificate, the union of two certificates, and insertion of a new justified pair. It then constructs `BoundCodeRel` from the resulting certificate and extracts naturalness from a certified pair.

Earlier constructor views and unique-existence induction simplify inversion and global functionality. They do not, by themselves, supply this local union/insertion algebra. A small generic interface can isolate it.

6.2 An exact abstract interface

Let `Item` and `Cert` be arbitrary types. Supply membership, an empty certificate, union, and singleton, together with their usual membership specifications. Let

$$\mathrm{Good}(x), \qquad \mathrm{Step}(S, x)$$

be predicates. Require only monotonicity of the step predicate:

$$S \subseteq T \ \wedge \ \mathrm{Step}(S, x) \implies \mathrm{Step}(T, x).$$

Define

$$\mathrm{Closed}(S) :\Longleftrightarrow \forall x \in S, \ \mathrm{Good}(x) \wedge \mathrm{Step}(S, x).$$

Then four elementary theorems provide the source-level API:

1. The empty certificate is closed.
2. If S and T are closed, their union is closed.
3. If S is closed, $\mathrm{Good}(x)$ holds, and $\mathrm{Step}(S, x)$ holds, then $S \cup \{x\}$ is closed.
4. A certified member of a closed certificate satisfies `Good`.

All are proved directly by the membership specifications. The supplied generic Lean development checks with no axioms at all.

6.3 Mapping it to the actual model

For the bound-code application set

$$\mathrm{Item} = V \times V, \qquad \mathrm{Cert} = V,$$

$$(c, b) \in_* S \quad \Longleftrightarrow \quad \mathrm{mem}(\mathrm{kpair}_H(c, b), S),$$

$$\mathrm{Good}(c, b) \Longleftrightarrow b \in \omega^H, \qquad \mathrm{Step}(S, (c, b)) \Longleftrightarrow \mathrm{BoundStep}_H(S, c, b).$$

The union and singleton specifications follow from `vcup_spec`, `vsingle_spec`, and injectivity of `kpair`. Non-pair members of an internal set remain ignored, exactly as they are by the current `BoundClosed` definition.

There is a genuine adapter detail: `boundStep_mono` is currently stated for inclusion of all set members, whereas the abstract interface above sees only encoded-pair membership. Its proof can be restated under pair-only inclusion because every occurrence of S in `BoundBinary` and `BoundQuantifier` is membership of an encoded pair. The old full-subset theorem can remain as a wrapper. This adapter has not been integrated or checked against the full object-module environment here.

6.4 Limits and the adoption decision

Monotonicity of `Step` does *not* make `Closed` upward-closed under arbitrary enlargement: an added member may fail its local justification. The insertion theorem requires that justification explicitly. These lemmas also do not establish a least fixed point, well-foundedness, totality, functionality, or the absence of circular supporting graphs. Those mathematical properties remain with their existing proofs.

No model-set certificate is assumed externally finite. No arbitrary internal code is replaced by a standard Lean syntax tree. The abstraction works with the membership interface of the original model itself.

Only one direct current client has been established in this review. The generic module and adapter could cost more source than the local facts they replace. Therefore this is a low-priority design: adopt it when a second real internal certificate relation needs the same union/insertion algebra, or when it substantially clarifies the module boundary. It is not included in any promised net line saving. The checked prototype is useful now as a precise specification of that future boundary.

7 Integration sequence and evidence discipline

7.1 A practical order

First: discriminant transport. Keep the existing `ShortEquation` residue theorem. Replace the three equivalent obstruction proofs by the checked specializations. This has a small semantic surface and preserves the four signatures.

Second: heterogeneous minimum saturation. Replace the `minima76` extraction proof with one finite-sum equality application. Keep the existing `minima75` specialization, which already inherits this result. Keep the coupled block intact.

Third: remove the redundant internal inductions. Prefer the small `NaturalCases` import extraction over importing the full satisfaction layer into basic arithmetic. Check the two source patches together and rebuild their real dependency closure. The necessary totality and uniqueness facts remain visible in the replacement.

Fourth: Pell foundation relocation and negative obstruction. Move the existing fundamental-unit code once, prove the first-coordinate congruence, and install the checked arithmetic replacement. Inspect imports and axiom sets as well as source size.

Later: certificate closure. Wait for a second concrete client or a demonstrated architectural benefit. The generic theorem is already proved; the decision is whether its adapter cost earns its place.

7.2 What the validation artifacts mean

A standalone proof that copies a source expression is useful evidence, but it does not automatically validate the complete module containing that expression. Conversely, a full-module replay may use unchanged lower modules recompiled from the snapshot without constituting a build of every project endpoint. The receipts distinguish these cases.

The full modified `MinimumNorm` replay checks its downstream finalizer theorems as well as the changed local statements. The separate Pell module replay checks the shared unit module and negative-norm replacement. Neither replay constitutes a build of every consumer.

The checks in this archive establish arithmetic correctness of the Pell replacement, the obstruction specializations, and the block-minimum mechanism; they also establish the abstract semantic correctness of the new case/closure interfaces. Exact source patches are provided where practical. The object-model import extraction and the higher native integration of the already checked Pell modules remain reviewable integration work.

No source reduction is credited to deleting comments, compressing formatting, removing checks, weakening a hypothesis, changing a definitional program value, or importing an unproved assertion. There are no new axioms or admitted theorem bodies in the checked prototypes. The logs record the actual axiom reports. Moving an existing classical theorem into the proof path may still alter an endpoint's axiom dependencies, so the production endpoint audit should compare them explicitly.

7.3 Measurements and counterfactual honesty

The baseline source files and their SHA-256 hashes are included in a manifest, together with proof-level receipts. Observed line counts are local comparisons against the pinned repository. They do not prove that applying all five recommendations produces their sum as a reduction: two touch the same module, some old proposals would already shorten portions of a proof, and the optional certificate interface adds a shared module.

The most reliable incremental measures here are structural: two unnecessary inductions disappear; one independent Pell descent disappears; three duplicate obstruction certificates become adapters; five nonlinear equality extractions become one finite-sum theorem. Those changes still make sense after the earlier roadmap has been implemented.

8 Artifact guide and reproduction

The ZIP contains this LaTeX source and its compiled PDF, checked Lean prototypes, exact source patches and replay overlays, the prior-proposal audit, a corpus manifest, selected-source hashes, validation logs, and reproduction instructions.

Use the included `README.md` for the exact file inventory and the `validation/` receipts for the final validation scope. The prototype runner supplied for reproduction is independent of the temporary paths used during this session: invoke it from a checkout configured with Lean 4.32.0 and the pinned mathlib revision, or follow the listed `lakeenvlean` commands. The PDF can be rebuilt with two runs of `pdflatex-halt-on-errornew-proof-simplifications.tex` in its report directory.

The source files are linked above at immutable commit URLs. The relevant primary references are:

1. ProveIt snapshot `9bd29079d5e1c6cc1c0bcaee6cb57d59b835cba9`.
2. The six placed proof-simplification packets, including their proposal notes and adoption ledger.
3. The five incoming archives at the same snapshot.
4. Mathlib's Pell solution group, fundamental-unit classification, and multiplication laws at the pinned revision.
5. Mathlib's finite-sum equality-at-bounds theorem at the pinned revision.